RAIDFLASH PD

Ocupación de prefill y decode. Un trabajo llena el die.

Jonathan Briceño De Luise · Peruano de nacimiento, 1982 · octubre 2026 · v1.0

Resumen

Los engines de serving ya abren N caminos concurrentes. Los techos publicados son tokens a alta concurrencia (StorageReview 2×GB300 Flash: 353 tok/s a un stream, 2721 a 32). RAIDFLASH asigna esos caminos a un trabajo — o, si hay varias personas, parte el die lleno. Prefill (P) y decode (D) comparten el mismo pool de asientos y tienen codos distintos.

En 2×GB10 TensorFold GLM-5.3-Flash EXL3 TP2, tokens de `usage` con `delta==sum`:

• D: 49.5 → 121.1 tok/s a k=8 (2.45× vanilla C1).
• P-hot: seed drafted (`max_tokens ≥ 16`) y k hermanas, cache ≥98.7%, 5.94× contra seeds seriales; packer de un acta 8k 94.6 tok/s, 2064 tokens en 21.8 s.
• P-cold: ~1600 tok/s ya a k=1 (GEMM lleno). Ocho prompts 8k distintos: 2317 → 8806 tok/s.

Vanilla en esta caja es un stream (~50 tok/s) con siete asientos vacíos. RAIDFLASH 1.0 es ocupación, no un kernel nuevo. Se copia la ley, no el 8.

```
k = min(PARALLEL, k_codo_fase)
```

1. De dónde sale

Esto no nace en un laboratorio de NVIDIA. Sale de una persona mirando una GPU cara mientras un humano esperaba la respuesta.

El mercado ya sabía paralelizar. La cifra que se celebra es throughput del cluster. La cifra que ve el humano es tokens de un cursor. RAIDFLASH junta esas dos cifras: si el engine abre N caminos para N inquilinos, esos mismos caminos pueden ser las tiras de un trabajo.

Autor: Jonathan Briceño De Luise, peruano de nacimiento, 1982.

No es un fork de TensorFold ni un kernel de atención nuevo. Es la política de para quién corren las lanes.

2. Ley P y D

`PARALLEL` es el tope de asientos. `k_codo` es el último k que todavía sube el agregado. Asientos ociosos bajo el codo son un fallo. Asientos pasado el codo, también.

fase	qué limita	k en este GB10
P-cold	compute / GEMM	1 por documento
P-hot	KV tras un seed drafted	8 hermanas, mismo prefijo, ~99% cache
D	memoria / recarga de expertos	8 (2.45× C1)

La profundidad DFlash (`most=7`) es profundidad de una cadena, no número de lanes. A k=8, `MULTI_WINDOW=32` deja 4 filas/stream: most cabe como 4. Thinking corto (`max_tokens ≤ 80`) usa k=4 para conservar most=7.

Inyector en 8 asientos:

gente	D / P-hot	P-cold
1	`[8]`	`[1]`
3	`[3, 3, 2]`	`[1, 1, 1]`

Un mapa B300 puede pedir 32 / 64 / 120. Se mide. El 8 de GB10 no se copia a HBM.

3. Cómo se parte un trabajo

1. Seed (P). Se prellena el documento una vez. `max_tokens ≥ 16` para que el keep de DFlash sea resumible. Seed mt=8 → 0% cache en hermanas (medido).
2. Hermanas (P-hot + D). k streams: mismo prefijo + «sección i». El prefijo pega en cache. El decode de las k tiras corre junto.
3. Fusión. Un `POST /v1/raidflash/complete`. El cliente ve una completion. El GPU vio k streams.

P-hot es KV, no especulación. El GEMM corre o el prefijo ya está.

4. Este engine (EFI, `delta==sum`)

2× NVIDIA GB10 Spark, GLM-5.3-Flash EXL3, TensorFold CUDA, `--parallel 8`, `PREFILL_ROWS=4096`, `FILL_ROWS=2048`, pool ~2.03M, árbol OFF.
Imagen viva `raidflash-glm53:1.0.4`, `RAIDFLASH_PREFIX=0`.

Decode (EFI cortos)

k	agg tok/s	vs C1
1	49.5	1.00×
2	71.0	1.43×
4	81.1	1.64×
8	121.1	2.45×

Prefill frío

k	agg prefill tok/s
1	1586
8	1629 (plano)

Ocho prompts ~8k independientes: 2317 → 8806 tok/s.

Prefill caliente (seed + hermanas, mt=16)

k	cache min	vs seeds seriales
8	99.0%	5.94× (5.40 s vs ~32 s)

Packer acta 8k, stripe 256 (Engine 1.0.4)

completion	2064 (seed 16 + 8×256)
cache	98.7%

completion	2064 (seed 16 + 8×256)
wall	21.8 s
agg	94.6 tok/s
prefill hermanas	0.83 s tras seed 3.90 s

Los `completion_tokens` de hermana son tokens generados, no el `accepted` de DFlash. Contar `accepted` subreportaba ~150 tok/lane (agg 62). Corregido en 1.0.4.

5. Segundos y tokens contra vanilla C1

Vanilla = un stream en el mismo silicio. Un chat Hermes corto sigue siendo 1 lane (~50 tok/s). La ocupación paga cuando el trabajo llena asientos.

trabajo	vanilla C1	RAIDFLASH PD	ganancia
1024 prosa	30.2 s · 33.9 tok/s	12.8 s · 80.3 tok/s (8×128)	−17.4 s
2048 D	~37 s @ 55 tok/s	16.9 s @ 121 tok/s	−20 s
5000 prosa	44 s, corta en 1590 tok	56.5 s, 5000 tok (8×625)	+3410 tok
acta 8k, 8×256	~45 s serial	21.8 s · 94.6 tok/s	−23 s
8 prefills fríos 8k	~2317 tok/s	8806 tok/s	~3.8× prompt
código 1024	57 tok/s	109 tok/s	+52
thinking 256	45 tok/s	115 tok/s	+70

Mismo presupuesto 1024: occupancy emitiría ~2400 tokens en los 30 s de vanilla.

6. Producto 1.0

Ocupación in-engine: `POST /v1/raidflash/complete`, cada chat es un `ChatJob` (`job_id`), tabla EFI por máquina (`raidflash probe`), métricas `raidflash:*`, cola que prioriza jobs. Overlay quirúrgico TensorFold (`DepthPolicy.most`, nunca pisar `fnc` con `fc7`). Flag off = engine de stock.

Receta que pagó: `--parallel 8`, pool KV gordo, `MULTI_WINDOW=32`, seed ≥16, RAIDFLASH-D most=7 desfijado.

Falsado aquí (queda off): árbol T3, P‖D a nivel request (−59% D), `MULTI_WINDOW=64`, k=16 por defecto (bajó el pico D), seed mt=8 (0% cache), prefix-kqi con SMEM de más en GB10 (D 107→83; solo lab).

MLA / Hydragen es Dynamo, fuera de esta pila. Vecino: IPPD (arXiv 2609.05707).

CAKE es el producto que le responde a gente. RAIDFLASH es la capa para que esa respuesta gaste el chip que se pagó.

7. Cierre

Los métodos de este v1.0 son toscos a propósito: un seed, k hermanas, una tabla EFI, un POST. El kernel fino lo escribe la comunidad. El golpe es el permiso:

usa el GPU que ya tienes, al 100%.

Quien iba a comprar otra tarjeta puede gozar primero la que ya pagó.
AI soberano. AI local. CAKE.

English: PAPER-PD.md. Tablas: TABLE.md.
Ley: OCCUPANCY.md. Mapa: [EFI.md](EFI.md).

Apache-2.0. Solo overlay — TensorFold no se vende dentro.